Ejercicio de Aplicación 🤓

L7: Optimización de un modelo

Es hora de que pongas en práctica todo lo aprendido. 🤓

Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

   A partir de un dataset desbalanceado de predicción binaria (por ejemplo, detección de fraude o churn), deberás ***construir un pipeline completo en Scikit-learn*** *que contemple:*

   - *Preprocesamiento de datos (normalización).*
   - *Aplicación de técnicas de Feature Engineering (codificación y selección de variables).*
   - *Entrenamiento de un modelo base (puedes usar LogisticRegression o RandomForestClassifier).*
   - *Optimización de hiperparámetros con GridSearchCV.*
   - *Regularización (si aplica).*
   - *Balanceo de clases usando class_weight='balanced' o técnicas como SMOTE (si lo deseas).*
   - *Evaluación del desempeño (precisión, recall y F1-score).*

2. **¿Dónde se lleva a cabo? 👩‍💻**

   Visual Studio Code

3. **Tiempo de dedicación ⏳**

   2 Horas.

4. **Recursos🛠️**
   - https://scikit-learn.org/stable/
   - https://imbalanced-learn.org/stable/
   - https://scikit-learn.org/stable/datasets/toy_dataset.html

5. **Plus ➕**

   Comparar el rendimiento entre un modelo optimizado y otro sin optimización.

6. **⚠️Condición**

   Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Ejercicio de Aplicación 🤓

L7: Optimización de un modelo

Resolución del desafío 👇

```python
# 1. Importar librerías necesarias
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Cargar el dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)

# 3. Dividir datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# 4. Crear pipeline con escalado + modelo
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(class_weight='balanced', solver='liblinear'))  # regularización L1/L2
])

# 5. Definir grilla de hiperparámetros
param_grid = {
    'clf__C': [0.01, 0.1, 1, 10],  # fuerza de la regularización
    'clf__penalty': ['l1', 'l2']   # tipo de regularización
}

# 6. Aplicar GridSearchCV
grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='f1', n_jobs=-1)
grid_search.fit(X_train, y_train)

# 7. Evaluar en el set de prueba
y_pred = grid_search.predict(X_test)

print("🔍 Mejores hiperparámetros encontrados:")
print(grid_search.best_params_)

print("\n📊 Reporte de clasificación:")
print(classification_report(y_test, y_pred, target_names=data.target_names))

# 8. Matriz de confusión
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=data.target_names, yticklabels=data.target_names)
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.title('Matriz de Confusión')
plt.show()
```